WEEK-1 DSA REFRESHER

Day 2

Calculating time complexities:

No loop	1
One loop over all items	n
A loop inside another loop	n^2
Number that keeps doubling/halving inside a loop	log n

Big -O notation:

It is defined as the condition that allows an algorithm to complete statement execution in the longest amount of time possible. It represents the ***upper bound*** of the running time of an algorithm.

- It is the most widely used notation
- It specifies the upper bound of a function
- Worst-case time complexity of an algorithm
- Returns the highest possible output value for a given input

Omega notation:

It is defined as the condition that allows an algorithm to complete statement execution in the shortest amount of time possible. It represents the ***lower bound*** of the running time of an algorithm.

- The execution time serves as a lower bound on the algorithm's time complexity
- It provides the best case time complexity of an algorithm

Theta notation:

It encloses the function from above and below. You add the running times for each possible input combination and take the average in the ***average case***.

- It is used for analysing the ***average-case complexity*** of an algorithm.
- It represents the ***upper and lower bound*** of the running time of an algorithm.

Time complexity:

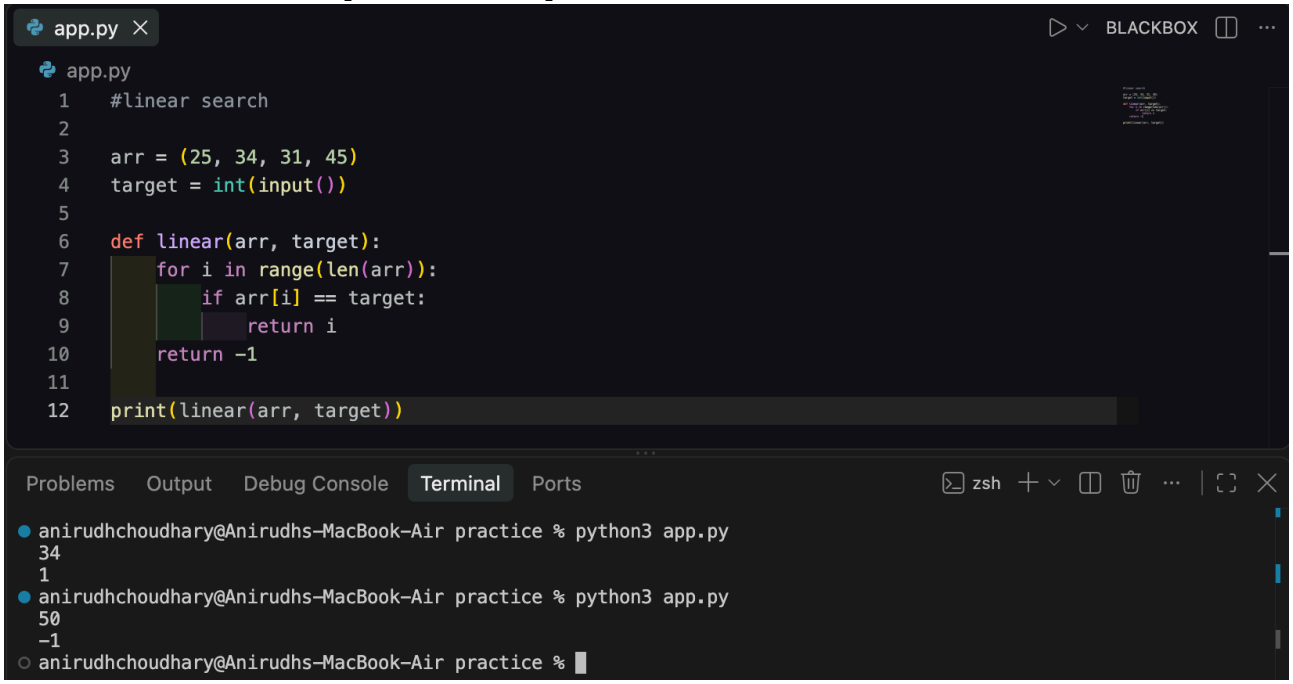
Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run, as a function of the length of the input. ***The time to run is a function of the length of the input and not the actual execution time of the machine on which the algorithm is running on.***
The valid algorithm takes a finite amount of time for execution.

Space complexity:

The amount of memory required by the algorithm to solve a given problem is called space complexity. Problem-solving using computer requires memory to hold temporary data while the program is in execution.
The space complexity quantifies the amount of space taken by an algorithm to run, as a function of the length of the input.

Only a few variables	1
New list, set, dict up to n items	n
A copy of the input {arr[:], sorted(arr)}	n
New nXn grid	n^2
Recursion n calls deep	n
Recursion that halves the input each time	Log n

Linear search: practice problem



Time complexity = O(n)
Space complexity = O(1)

Finding minimum and maximum:

```
#finding minimum and maximum

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9]

def find(arr):
  small, big = arr[0], arr[0]
  for i in arr[1:]:
    if i < small:
      small = i
    elif i > big:
      big = i
  return small, big
small, big = find(arr)
print("minimum element: ", small)
print("maximum element: ", big)
```

```
minimum element:  1
maximum element:  9
```

Time complexity = O(n)
Space complexity = O(n)

Reversing a list:

```
#reversing a list

list = [10, 11, 12, 13, 14, 15]
print("list before reversing: ", list)

list.reverse()
print("list after reversing: ", list)
```

```
list before reversing:  [10, 11, 12, 13, 14, 15]
list after reversing:  [15, 14, 13, 12, 11, 10]
```

Time complexity = O(n)
Space complexity = O(1)

Removing duplicates:

```
#removing duplicates

list = [1, 1, 2, 2, 3, 3, 3, 8, 7, 8, 0, 2, 0]
final_list =[]

for i in list:
  if i not in final_list:
    final_list.append(i)

print(final_list)
```

```
[1, 2, 3, 8, 7, 0]
```

Time complexity = O(n^2)
Space complexity = O(n)